

Student Tasks

Final Project Brief

You are building a compact computer vision system for a small fashion platform. The platform needs a model that can:

- classify clothing items,
- retrieve visually similar items,
- run efficiently on limited hardware.

This is one end-to-end project with several connected stages. Each stage should build on the previous one.

The intended dependency between parts is:

- Part A selects the best Part A backbone.
- Part B reuses the selected Part A model as the initialization for retrieval training.
- Part C compresses the Part B retrieval model.

To keep the project reproducible and lightweight, the architecture scaffold is fixed. The standard CNN is provided in `src/models.py`, and you must implement the depthwise separable version inside the provided scaffold without changing the overall architecture.

Use the provided shared data pipeline in `src/data.py` so that experiments remain comparable across students.

Required Project Stages

Part A: Build And Compare Two Backbones

Train and compare the following two classifiers on FashionMNIST:

- a standard CNN,
- a depthwise separable CNN.

For reproducibility:

- StandardCNN is already implemented for you in `src/models.py`,
- you must implement the missing depthwise separable components in

SeparableCNN,

- you must not change the number of blocks, channel sizes, pooling pattern, or classifier width.

The point of Part A is to isolate the effect of replacing standard convolutions with depthwise separable convolutions under the same small architecture budget.

Suggested background reference: [François Chollet, "Xception: Deep Learning With Depthwise Separable Convolutions," CVPR 2017.](#)

You must report:

- accuracy,
- macro F1,
- parameter count,
- inference efficiency, reported as FLOPs for one forward pass on a single input.

The output of this part must be one selected Part A backbone that you will reuse in Part B.

Part B: Learn Embeddings For Retrieval

Take the backbone you selected in Part A, replace the classification head with an embedding head, and train that model for retrieval using either:

- contrastive loss, or
- triplet loss.

Your goal is to support image retrieval:

- given one fashion item,
- return visually similar items.

You should initialize Part B with the trained weights of the selected Part A model.

You should explain whether the embedding space is useful for retrieval.

For retrieval quality, report at least a top-1 retrieval metric, for example Recall@1.

Suggested background references:

- [Raia Hadsell, Sumit Chopra, and Yann LeCun, "Dimensionality Reduction by Learning an Invariant Mapping," CVPR 2006.](#)

- [Florian Schroff, Dmitry Kalenichenko, and James Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," CVPR 2015.](#)

Part B should also include:

- a script that visualizes retrieval examples for a query image and its nearest neighbors,
- a script or function that visualizes the embedding space, for example with t-SNE.

Part C: Compress The Retrieval Model

Take the Part B retrieval model and study both of the following compression methods:

- pruning,
- knowledge distillation into the fixed CompactSeparableCNN student in `src/models.py`.

Both pruning and knowledge distillation are required in Part C. They are two separate subsections of the same stage, not two alternatives.

For pruning, you are free to choose the pruning method, as long as you justify the choice and evaluate its effect clearly.

Part C should keep the retrieval objective from Part B. In other words, the Part B embedding model is the teacher, and the distilled student should still produce useful retrieval embeddings.

You should explain:

- how much retrieval quality is preserved after pruning,
- how well the distilled student retains retrieval performance,
- whether the compression tradeoff is worthwhile for deployment.

Your final discussion should explain the tradeoff between:

- retrieval usefulness,
- deployment efficiency in terms of FLOPs and number of parameters.

Report Questions

- Which backbone would you keep after comparing CNN and separable CNN, and why?
- Did your retrieval model learn meaningful similarity?

- How much performance was preserved after pruning and distillation?
- What final model would you deploy, and why?

Expected Submission

Submission deadline is June 5th.

Submit:

- Python source files only,
- a short report including answers to the above questions,
- a results table,
- trained checkpoints,
- clear run instructions.

Evaluation Notes

Your evaluation is not based only on whether the code runs or on the absolute performance numbers you obtain.

Important factors also include:

- whether your evaluation protocol is sound and clearly explained,
- whether the results in your report are interpreted carefully and technically,
- whether your final conclusions are consistent with the experiments you actually ran.

There will also be a short oral question-and-answer component of about 5 minutes for this project.

Important Note

This package does not give you a full starter solution. The fixed backbone scaffold is provided for reproducibility, but you are still expected to implement the missing separable components and the training, evaluation, retrieval, and compression workflow yourself.

However, the dataset protocol is fixed. You should use the provided shared data-loading utilities so that all students work with the same splits and subset sizes.